

From your description and the whiteboard, I think the OA problem was approximately this.

---

## Maximum Memory Collection with Row Limit

**Difficulty:** Hard

You are given an  $n \times m$  integer matrix `memory`, where `memory[i][j]` represents the amount of memory that can be collected from cell `(i, j)`.

You start at the top-left cell `(0, 0)` and want to reach the bottom-right cell `(n - 1, m - 1)`.

At every cell, you may perform exactly one of the following actions:

1. **Read** the current cell and collect its memory value.
2. **Skip** the current cell and collect nothing.

After that, you must move either

- one cell to the **right**, or
- one cell **down**,

provided the move stays inside the matrix.

### Constraint

For every row, you may **read** at most `k` cells from that row.

Return the **maximum total memory** that can be collected.

---

### Example 1

#### Input

```
memory =  
[  
  [4, 5, 9],  
  [1, 2, 4],  
  [6, 5, 2]  
]  
  
k = 2
```

#### Output

21

## Explanation

One optimal path is

$(0,0) \rightarrow (0,1) \rightarrow (1,1) \rightarrow (1,2) \rightarrow (2,2)$

Read

4, 5, 2, 4, 2

Total

17

However, a better path is

$(0,0) \rightarrow (1,0) \rightarrow (2,0) \rightarrow (2,1) \rightarrow (2,2)$

Read

$4 + 6 + 5 + 2 = 17$

An optimal strategy gives a total of **21** while never reading more than **2** cells from any row.

---

## Example 2

```
memory =  
[  
  [5,1],  
  [3,7]  
]  
  
k = 1
```

Output

12

Explanation

Read

5

from the first row and

7

from the second row.

---

## Function Signature

C++

```
class Solution {  
public:  
    long long maximumMemory(vector<vector<int>>& memory, int k);  
};
```

---

## Constraints

$1 \leq n, m \leq 200$

$1 \leq \text{memory}[i][j] \leq 10^9$

$1 \leq k \leq m$

---

## Expected DP

From your whiteboard, I can also infer the intended DP.

State

C++

```
dp(i, j, cnt)
```

where

- `i` = current row
- `j` = current column

- `cnt` = number of cells already **read** in the current row

Notice **cnt resets to 0 whenever we move down.**

---

At every cell there are **4 choices.**

## 1. Read + Right

C++

```
if(cnt < k)
ans = memory[i][j] + dp(i, j+1, cnt+1);
```

---

## 2. Skip + Right

C++

```
ans = dp(i, j+1, cnt);
```

---

## 3. Read + Down

C++

```
if(cnt < k)
ans = memory[i][j] + dp(i+1, j, 1);
```

because we start a new row and have already read one cell there.

---

## 4. Skip + Down

C++

```
ans = dp(i+1, j, 0);
```

---

So

C++

```
dp(i, j, cnt)
=
max(
  Read+Right,
  Skip+Right,
  Read+Down,
  Skip+Down
)
```

---

## Complexity

States

$O(n \times m \times k)$

Each state has constant transitions.

So

Time :  $O(nmk)$

Space :  $O(nmk)$

---

## One thing I noticed from your whiteboard

The notation at the bottom looked like

$f(i, j, k)$

with arrows to

$f(i, j+1, \dots)$

$f(i+1, j, \dots)$

and the notes "**take / not take**".

That strongly matches this DP formulation, where the third parameter is the number of values already picked in the current row.

This is a clean interview DP problem because the key insight is that **the row constraint only depends on the current row**, so the state doesn't need to remember anything about previous rows once you move down.

This formulation is exactly what I'd expect for a Harness OA question.